



INFORME DE LA GUÍA PRÁCTICA

I. PORTADA

Tema:	Arboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero – B
Alumnos participantes:	Poaquiza Ayme Gilberth Joel
Asignatura:	ESTRUCTURA DE DATOS
Docente:	Ing. Jose Caizabuano, Mg.

II. INFORME DE GUÍA PRÁCTICA

Objetivos

General:

Implementar y validar algoritmos clásicos sobre estructuras de árbol —N-ario y binario— en C++ y Java, verificando su corrección mediante casos de prueba predefinidos.

Específicos:

- Comprender la diferencia estructural entre un árbol N-ario y un árbol binario de búsqueda (BST).
- Implementar operaciones de conteo, inserción, cálculo de altura, recorrido y transformación mediante recursividad.
- Verificar la exactitud de cada implementación comparando la salida calculada contra los resultados esperados definidos en el método main.
- Desarrollar código limpio, comentado y fiel a las restricciones del enunciado (modificar únicamente la sección TODO).

Modalidad

Presencial

Instrucciones

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC
- Apuntes de clase
- Internet
- Dev – C++

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación



☒ Inteligencia Artificial
Otros (Especifique): _____

Actividades por desarrollar

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

Resultados obtenidos

1. Introducción

La presente práctica tiene como propósito consolidar los conocimientos teóricos sobre estructuras de datos no lineales, específicamente los árboles. A través de cinco ejercicios progresivos, se implementaron algoritmos fundamentales en dos lenguajes de programación —C++ y Java— aplicando el principio de recursividad como mecanismo central de procesamiento.

Los árboles constituyen una de las estructuras de datos más versátiles en ciencias de la computación, siendo la base de motores de búsqueda, sistemas de archivos, compiladores y bases de datos. Dominar su manipulación es, por tanto, una competencia esencial en la formación de un ingeniero de software.

2. Objetivos

2.1 Objetivo General

Implementar y validar algoritmos clásicos sobre estructuras de árbol —N-ario y binario— en C++ y Java, verificando su corrección mediante casos de prueba predefinidos.

2.2 Objetivos Específicos

- Comprender la diferencia estructural entre un árbol N-ario y un árbol binario de búsqueda (BST).
- Implementar operaciones de conteo, inserción, cálculo de altura, recorrido y transformación mediante recursividad.
- Verificar la exactitud de cada implementación comparando la salida calculada contra los resultados esperados definidos en el método main.
- Desarrollar código limpio, comentado y fiel a las restricciones del enunciado (modificar únicamente la sección TODO).

3. Descripción de los Ejercicios y Propuestas de Solución

3.1 Ejercicio 1 – Conteo de Nodos en Árbol N-ario

Objetivo: Implementar la función contarNodos() que retorne el número total de nodos de un árbol N-ario dado su nodo raíz.

Aspectos fundamentales de la solución:

- Caso base: si el puntero/referencia es nulo, se retorna 0 para detener la recursión.
- Paso recursivo: se inicializa un contador en 1 (el nodo actual) y se itera sobre la lista de hijos, acumulando el resultado de llamadas recursivas a cada uno.
- Esta estrategia de suma bottom-up garantiza que cada nodo se cuente exactamente una vez, independientemente de la ramificación del árbol.

Fragmento clave — C++:



```
Nodo* invertir(Nodo* raiz) {  
    // TODO: Implementa tu lógica aquí  
    if (raiz == nullptr) return nullptr;  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = invertir(raiz->derecho);  
    raiz->derecho = invertir(temp);  
    return raiz;  
}
```

Fragmento clave en Java

```
public class Ejercicio1 {  
    public static int contarNodos(NodoN raiz) {  
        // TODO: Implementa tu lógica aquí. (Pista: usa recursividad)  
        if (raiz == null) return 0;  
        int total = 1;  
        for (NodoN hijo : raiz.hijos) {  
            total += contarNodos(hijo);  
        }  
        return total;  
    }  
}
```

Resultado obtenido: 6 nodos (coincide con el esperado).

Resultado en C++

```
--- Prueba Ejercicio 1 ---  
Nodos esperados: 6  
Nodos calculados: 6
```

Resultado en java

```
--- Prueba Ejercicio 1 ---  
Nodos esperados: 6  
Nodos calculados: 6
```

3.2 Ejercicio 2 – Inserción en Árbol Binario de Búsqueda (BST)

Objetivo: Implementar insertar() que ubique un valor en la posición correcta de un BST respetando la propiedad de orden: menores a la izquierda, mayores o iguales a la derecha.

Aspectos fundamentales de la solución:

- Se utiliza recursión estructural: si el nodo actual es nulo, se crea y retorna un nuevo nodo con el valor dado.
- La comparación `valor < raiz->valor` dirige el descenso hacia el subárbol izquierdo o derecho según corresponda.
- El retorno del puntero (o referencia) en cada nivel de recursión es fundamental para que los nuevos nodos queden correctamente enlazados en el árbol padre.

Fragmento clave — C++:

```
Nodo* insertar(Nodo* raiz, int valor) {  
    // TODO: Implementa tu lógica aquí  
    if (raiz == nullptr) return new Nodo(valor);  
    if (valor < raiz->valor)  
        raiz->izquierdo = insertar(raiz->izquierdo, valor);  
    else  
        raiz->derecho = insertar(raiz->derecho, valor);  
    return raiz;  
}
```

Fragmento clave en Java



```
public class Ejercicio2{
    public static Nodo insertar(Nodo raiz, int valor) {
        // TODO: Implementa tu lógica aquí
        // Recuerda: menores a la izquierda, mayores o iguales a la derecha.
        if (raiz == null) return new Nodo(valor);
        if (valor < raiz.valor)
            raiz.izquierdo = insertar(raiz.izquierdo, valor);
        else
            raiz.derecho = insertar(raiz.derecho, valor);
        return raiz;
    }
}
```

Resultado obtenido: raíz=10, hijo izquierdo=5, hijo derecho=15, hijo izquierdo de 5=3 (todos correctos).

Resultado en C++

```
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
```

Resultado en java

```
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo de Raiz (Esperado 5): 5
Hijo Derecho de Raiz (Esperado 15): 15
Hijo Izquierdo del 5 (Esperado 3): 3
PS C:\Users\JOEL\Downloads\Ape 3>
```

3.3 Ejercicio 3 – Cálculo de Altura Máxima

Objetivo: Implementar calcularAltura() que retorne la profundidad máxima del árbol, donde un árbol nulo tiene altura 0 y uno con un solo nodo tiene altura 1.

Aspectos fundamentales de la solución:

- El caso base nulo retorna 0, satisfaciendo simultáneamente el árbol vacío y el límite de la hoja.
- La expresión $1 + \max(\text{altura_izquierda}, \text{altura_derecha})$ captura de forma elegante la definición formal de altura: la longitud del camino más largo desde la raíz hasta una hoja.
- El uso de `std::max` (C++) y `Math.max` (Java) evita ramificaciones condicionales innecesarias.

Fragmento clave — C++:

```
int calcularAltura(Nodo* raiz) {
    // TODO: Implementa tu lógica aquí
    if (raiz == nullptr) return 0;
    return 1 + max(calcularAltura(raiz->izquierdo), calcularAltura(raiz->derecho));
}
```

Fragmento clave en Java



```
public class Ejercicio3 {
    public static int calcularAltura(Nodo raiz) {
        // TODO: Implementa tu lógica aquí
        if (raiz == null) return 0;
        return 1 + Math.max(calcularAltura(raiz.izquierdo), calcularAltura(raiz.derecho));
    }
}
```

Resultado obtenido: altura=3 para el árbol de prueba y 0 para árbol nulo (ambos correctos).

Resultado en C++

```
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de arbol nulo (esperado 0): 0
```

Resultado en java

```
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
```

3.4 Ejercicio 4 – Recorrido In-Order

Objetivo: Implementar inOrderAux() que llene una lista/vector con los valores del árbol en orden in-order (izquierdo → raíz → derecho).

Aspectos fundamentales de la solución:

- El caso base es implícito: si el nodo es nulo la función retorna sin realizar ninguna operación.
- El orden de las tres operaciones — visitar subárbol izquierdo, registrar el valor actual, visitar subárbol derecho — define el recorrido in-order y no debe alterarse.
- En un BST, este recorrido produce los elementos en orden ascendente, lo que permite validar la corrección de la inserción (Ejercicio 2).

Fragmento clave — C++:

```
void inOrderAux(Nodo* nodo, vector<int>& resultado) {
    // TODO: Implementa tu logica de recorrido aquí
    if (nodo == nullptr) return;
    inOrderAux(nodo->izquierdo, resultado);
    resultado.push_back(nodo->valor);
    inOrderAux(nodo->derecho, resultado);
}
```

Fragmento clave en Java

```
public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
    // TODO: Implementa tu lógica de recorrido aquí
    if (nodo == null) return;
    inOrderAux(nodo.izquierdo, resultado);
    resultado.add(nodo.valor);
    inOrderAux(nodo.derecho, resultado);
}
```

Resultado obtenido: [1, 2, 3, 4, 5, 6, 7] (coincide con el esperado).

Resultado en C++

```
--- Prueba Ejercicio 4 ---
Resultado esperado: 1 2 3 4 5 6 7
Tu resultado:      1 2 3 4 5 6 7
```



Resultado en java

```
--- Prueba Ejercicio 4 ---  
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]  
Tu resultado:      [1, 2, 3, 4, 5, 6, 7]
```

3.5 Ejercicio 5 – Inversión de Árbol (Árbol Espejo)

Objetivo: Implementar `invertir()` que transforme el árbol intercambiando recursivamente todos los subárboles izquierdo y derecho, produciendo el árbol espejo.

Aspectos fundamentales de la solución:

- El caso base nulo evita errores de desreferencia y detiene la recursión en las hojas.
- Se utiliza una variable temporal para preservar uno de los punteros antes de sobrescribirlo, evitando la pérdida de referencia.
- La inversión se realiza de abajo hacia arriba (post-order implícito): primero se invierten los subárboles hijos, luego se intercambian en el nodo actual, garantizando que el espejo sea completo.

Fragmento clave — C++:

```
Nodo* invertir(Nodo* raiz) {  
    // TODO: Implementa tu lógica aquí  
    if (raiz == nullptr) return nullptr;  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = invertir(raiz->derecho);  
    raiz->derecho = invertir(temp);  
    return raiz;  
}
```

Fragmento clave en Java

```
public static Nodo invertir(Nodo raiz) {  
    // TODO: Implementa tu lógica aquí  
    if (raiz == null) return null;  
    Nodo temp = raiz.izquierdo;  
    raiz.izquierdo = invertir(raiz.derecho);  
    raiz.derecho = invertir(temp);  
    return raiz;  
}
```

Resultado obtenido: hijo izquierdo=3, hijo derecho=2 (invertidos correctamente desde 2 y 3).

Resultado en C++

```
• --- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```

Resultado en java

```
--- Prueba Ejercicio 5 ---  
Antes de invertir:  
Hijo Izq: 2 | Hijo Der: 3  
  
Después de invertir (Esperado: Izq 3 | Der 2):  
Hijo Izq: 3 | Hijo Der: 2
```



4. Tabla Resumen de Soluciones

Ejercicio	Concepto Clave	Algoritmo	Complejidad
Ej. 1 – N-ario	Árbol N-ario / Conteo	Recursión sobre hijos	$O(n)$
Ej. 2 – BST Inserción	Árbol Binario de Búsqueda	Comparación y descenso	$O(\log n) / O(n)$
Ej. 3 – Altura	Profundidad máxima	Recursión + max()	$O(n)$
Ej. 4 – In-Order	Recorrido DFS	Izq $\rightarrow$ Raíz $\rightarrow$ Der	$O(n)$
Ej. 5 – Espejo	Transformación estructural	Swap recursivo de hijos	$O(n)$

5. Patrón Común de Solución: Recursividad

El principio unificador de los cinco ejercicios es la recursividad. Cada función sigue la misma estructura canónica:

- Caso base: condición sobre el nodo nulo (o hoja) que detiene la recursión y retorna un valor trivial.
- Paso recursivo: se descompone el problema en subproblemas más pequeños llamando a la misma función sobre los subárboles hijos.
- Combinación: el resultado parcial del nodo actual se combina con los resultados de los subproblemas para producir la respuesta final.

Esta estructura garantiza que la solución sea correcta para cualquier árbol válido, independientemente de su tamaño o forma, y es la base del análisis de complejidad: todas las soluciones implementadas tienen complejidad temporal $O(n)$, ya que cada nodo se visita exactamente una vez.

6. Consideraciones de Implementación: C++ vs. Java

La práctica requirió implementar cada algoritmo en ambos lenguajes. Los aspectos más relevantes de esta dualidad son

- Gestión de memoria: C++ utiliza punteros explícitos (NodoN*, Nodo*) con new, mientras que Java emplea referencias gestionadas automáticamente por el recolector de basura.
- Colecciones: C++ usa `std::vector<NodoN*>` para los hijos del árbol N-ario; Java usa `List<NodoN>` con `ArrayList`, proveyendo mayor abstracción.
- Funciones de apoyo: C++ requiere `std::max` de `<algorithm>`; Java usa `Math.max`, disponible sin importación adicional.
- Retorno de nodos: en C++ es necesario asignar el resultado de `insertar()` de vuelta (`raiz->izquierdo = insertar(...)`), pues el retorno del puntero actualizado es lo que conecta el nuevo nodo al árbol padre.



Habilidades blandas empleadas en la prueba práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

Conclusiones

- La recursividad es la herramienta más natural y eficiente para el procesamiento de estructuras arbóreas, dado que los árboles son estructuras inherentemente recursivas.
- El caso base sobre el nodo nulo es crítico: su omisión provoca errores de segmentación (C++) o NullPointerException (Java) en tiempo de ejecución.
- La propiedad de orden del BST, mantenida correctamente durante la inserción, habilita el recorrido in-order para producir secuencias ordenadas, lo que conecta lógicamente los Ejercicios 2 y 4.
- La inversión de árbol (Ejercicio 5) ilustra que las transformaciones estructurales también siguen el patrón recursivo, pero requieren especial atención al orden de las operaciones para no perder referencias.
- La implementación en dos lenguajes evidenció que los algoritmos son independientes del lenguaje; las diferencias se limitan a sintaxis, gestión de memoria y bibliotecas estándar.

Recomendaciones

Con base en la experiencia adquirida durante el desarrollo de esta práctica, se plantean las siguientes recomendaciones para el estudio y aplicación de estructuras de datos tipo árbol. En primer lugar, se recomienda dominar sólidamente el concepto de recursividad antes de abordar operaciones sobre árboles, ya que todos los algoritmos implementados —conteo, inserción, altura, recorrido e inversión— dependen de este principio; practicar con funciones recursivas simples (factorial, Fibonacci) facilita enormemente la comprensión posterior. En segundo lugar, es fundamental verificar siempre el caso base antes de escribir el paso recursivo: una función sin caso base correctamente definido produce desbordamiento de pila, error difícil de depurar en estructuras grandes. Se aconseja además trazar manualmente el árbol de prueba sobre papel antes de codificar, dibujando cómo debe quedar la estructura después de cada operación; esta práctica reduce significativamente los errores lógicos. Respecto a la gestión de memoria en C++, se recomienda liberar los nodos creados con `new` mediante `delete` al finalizar, o bien adoptar punteros inteligentes (`std::weak_ptr`) en proyectos de mayor envergadura para evitar fugas de memoria. En cuanto a la elección de estructura, conviene evaluar si un BST es la opción adecuada: si los datos de inserción llegan en orden, el árbol degenera en una lista enlazada con complejidad $O(n)$; en esos casos se deben preferir estructuras autoequilibradas como AVL o árboles rojo-negro. Finalmente, se recomienda ampliar la práctica con recorridos pre-order y post-order para consolidar la comprensión de los distintos órdenes de visita, y experimentar con la eliminación de nodos en un BST, operación que representa el siguiente nivel de complejidad natural tras la inserción.



Anexos:

Link Github: https://github.com/joel427/Guia-APE---3/tree/main/APE_Arboles

Bibliografía:

[1] P. A. Sznajdleder, *Programación orientada a objetos y estructura de datos a fondo*, 1a ed. Buenos Aires, Argentina: Alfaomega, 2012.

Uso de Inteligencia Artificial

- **Herramienta y Marco Normativo:** Se utilizó el modelo Claude como recurso de apoyo, cumpliendo con las disposiciones de la guía oficial de la asignatura.
- **Frente de Comprensión Conceptual:** La IA sirvió para validar la interpretación de las consignas, aclarando conceptos específicos como el recorrido *in-order* en árboles BST y la distinción entre altura y profundidad.
- **Frente de Implementación Asistida:** Se empleó para recibir retroalimentación sobre la lógica programada por el estudiante, permitiendo corregir errores técnicos en C++, tales como la omisión de casos base o retornos de punteros incorrectos.
- **Metodología de Trabajo:** El código no fue generado de forma automática; el estudiante propuso las soluciones y la herramienta funcionó como un revisor crítico de las mismas.
- **Objetivo Pedagógico:** Este enfoque buscó fortalecer el pensamiento crítico y el razonamiento algorítmico, priorizando el aprendizaje profundo sobre la mera obtención del resultado final.

Capturas del código:
Ejercicios en JAVA.

Ejercicio 1



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.List;
4
5  class NodoN {
6      public int valor;
7      public List<NodoN> hijos;
8
9      public NodoN(int valor) {
10         this.valor = valor;
11         this.hijos = new ArrayList<>();
12     }
13
14     public NodoN(int valor, List<NodoN> hijos) {
15         this.valor = valor;
16         this.hijos = hijos;
17     }
18 }
19
20 public class Ejercicio1 {
21     public static int contarNodos(NodoN raiz) {
22         // TODO: Implementa tu lógica aquí. (Pista: usa recursividad)
23         if (raiz == null) return 0;
24         int total = 1;
25         for (NodoN hijo : raiz.hijos) {
26             total += contarNodos(hijo);
27         }
28         return total;
29     }
30
31     Run | Debug
32     public static void main(String[] args) {
33         // Construimos el árbol:
34         //      1
35         //     / | \
36         //    2 3 4
37         //   / \
38         //  5  6
39         NodoN n5 = new NodoN(valor: 5);
40         NodoN n6 = new NodoN(valor: 6);
41         NodoN n2 = new NodoN(valor: 2, Arrays.asList(n5, n6));
42         NodoN n3 = new NodoN(valor: 3);
43         NodoN n4 = new NodoN(valor: 4);
44         NodoN raiz = new NodoN(valor: 1, Arrays.asList(n2, n3, n4));
45
46         System.out.println(x: "--- Prueba Ejercicio 1 ---");
47         System.out.println(x: "Nodos esperados: 6");
48         System.out.println("Nodos calculados: " + contarNodos(raiz));
49     }
50 }
```

Ejercicio 2



```
1 class Nodo {
2     int valor;
3     Nodo izquierdo, derecho;
4     Nodo(int valor) { this.valor = valor; }
5 }
6
7 public class Ejercicio2{
8     public static Nodo insertar(Nodo raiz, int valor) {
9         // TODO: Implementa tu lógica aquí
10        // Recuerda: menores a la izquierda, mayores o iguales a la derecha.
11        if (raiz == null) return new Nodo(valor);
12        if (valor < raiz.valor)
13            raiz.izquierdo = insertar(raiz.izquierdo, valor);
14        else
15            raiz.derecho = insertar(raiz.derecho, valor);
16        return raiz;
17    }
18
19    Run | Debug
20    public static void main(String[] args) {
21        Nodo raiz = new Nodo(valor: 10);
22
23        // Insertamos valores
24        insertar(raiz, valor: 5);
25        insertar(raiz, valor: 15);
26        insertar(raiz, valor: 3);
27
28        System.out.println(x: "--- Prueba Ejercicio 2 ---");
29        System.out.println("Raiz (Esperado 10): " + raiz.valor);
30
31        String izq = (raiz.izquierdo != null) ? String.valueOf(raiz.izquierdo.valor) : "null";
32        System.out.println("Hijo Izquierdo de Raiz (Esperado 5): " + izq);
33
34        String der = (raiz.derecho != null) ? String.valueOf(raiz.derecho.valor) : "null";
35        System.out.println("Hijo Derecho de Raiz (Esperado 15): " + der);
36
37        String izqIzq = (raiz.izquierdo != null && raiz.izquierdo.izquierdo != null) ? String.valueOf(raiz.izquierdo.izquierdo.valor) : "null";
38        System.out.println("Hijo Izquierdo del 5 (Esperado 3): " + izqIzq);
39    }
40 }
```

Ejercicio 3

```
1 public class Ejercicio3 {
2     public static int calcularAltura(Nodo raiz) {
3         // TODO: Implementa tu lógica aquí
4         if (raiz == null) return 0;
5         return 1 + Math.max(calcularAltura(raiz.izquierdo), calcularAltura(raiz.derecho));
6     }
7
8     Run | Debug
9     public static void main(String[] args) {
10        //      1
11        //      \
12        //      2
13        //      /
14        //      3
15        Nodo raiz = new Nodo(valor: 1);
16        raiz.derecho = new Nodo(valor: 2);
17        raiz.derecho.izquierdo = new Nodo(valor: 3);
18
19        System.out.println(x: "--- Prueba Ejercicio 3 ---");
20        System.out.println(x: "Altura esperada: 3");
21        System.out.println("Altura calculada: " + calcularAltura(raiz));
22        System.out.println("Altura de árbol nulo (esperado 0): " + calcularAltura(raiz: null));
23    }
24    // Nota: Usa la clase Nodo definida en el Ejercicio 2.
25 }
```

Ejercicio 4



```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Ejercicio4 {
5     public static void inOrderAux(Nodo nodo, List<Integer> resultado) {
6         // TODO: Implementa tu lógica de recorrido aquí
7         if (nodo == null) return;
8         inOrderAux(nodo.izquierdo, resultado);
9         resultado.add(nodo.valor);
10        inOrderAux(nodo.derecho, resultado);
11    }
12
13    public static List<Integer> recorridoInOrder(Nodo raiz) {
14        List<Integer> resultado = new ArrayList<>();
15        inOrderAux(raiz, resultado);
16        return resultado;
17    }
18
19    public static void main(String[] args) {
20        //      4
21        //    /  \
22        //   2   6
23        //  / \  / \
24        // 1  3 5  7
25        Nodo raiz = new Nodo(valor: 4);
26        raiz.izquierdo = new Nodo(valor: 2);
27        raiz.derecho = new Nodo(valor: 6);
28        raiz.izquierdo.izquierdo = new Nodo(valor: 1);
29        raiz.izquierdo.derecho = new Nodo(valor: 3);
30        raiz.derecho.izquierdo = new Nodo(valor: 5);
31        raiz.derecho.derecho = new Nodo(valor: 7);
32
33        System.out.println(x: "--- Prueba Ejercicio 4 ---");
34        System.out.println(x: "Resultado esperado: [1, 2, 3, 4, 5, 6, 7]");
35        System.out.println("Tu resultado: " + recorridoInOrder(raiz));
36    }
```

Ejercicio 5

```
1 public class Ejercicio5 {
2     public static Nodo invertir(Nodo raiz) {
3         // TODO: Implementa tu lógica aquí
4         if (raiz == null) return null;
5         Nodo temp = raiz.izquierdo;
6         raiz.izquierdo = invertir(raiz.derecho);
7         raiz.derecho = invertir(temp);
8         return raiz;
9     }
10
11    public static void main(String[] args) {
12        //      1
13        //    /  \  -->  1
14        //   2   3      /  \
15        //  2   3      3   2
16        Nodo raiz = new Nodo(valor: 1);
17        raiz.izquierdo = new Nodo(valor: 2);
18        raiz.derecho = new Nodo(valor: 3);
19
20        System.out.println(x: "--- Prueba Ejercicio 5 ---");
21        System.out.println(x: "Antes de invertir:");
22        System.out.println("Hijo Izq: " + raiz.izquierdo.valor + " | Hijo Der: " + raiz.derecho.valor);
23
24        invertir(raiz);
25
26        System.out.println(x: "\nDespués de invertir (Esperado: Izq 3 | Der 2):");
27        String izq = (raiz.izquierdo != null) ? String.valueOf(raiz.izquierdo.valor) : "null";
28        String der = (raiz.derecho != null) ? String.valueOf(raiz.derecho.valor) : "null";
29
30        System.out.println("Hijo Izq: " + izq + " | Hijo Der: " + der);
31    }
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  struct NodoN {
6      int valor;
7      vector<NodoN*> hijos;
8      NodoN(int v) : valor(v) {}
9  };
10
11 int contarNodos(NodoN* raiz) {
12     // TODO: Implementa tu lógica aquí
13     if (raiz == nullptr) return 0;
14     int total = 1; // contar el nodo actual
15     for (NodoN* hijo : raiz->hijos) {
16         total += contarNodos(hijo);
17     }
18     return total;
19 }
20
21 int main() {
22     // Construimos el árbol
23     NodoN* n5 = new NodoN(5);
24     NodoN* n6 = new NodoN(6);
25     NodoN* n2 = new NodoN(2);
26     n2->hijos.push_back(n5);
27     n2->hijos.push_back(n6);
28     NodoN* n3 = new NodoN(3);
29     NodoN* n4 = new NodoN(4);
30     NodoN* raiz = new NodoN(1);
31     raiz->hijos.push_back(n2);
32     raiz->hijos.push_back(n3);
33     raiz->hijos.push_back(n4);
34
35     cout << "--- Prueba Ejercicio 1 ---" << endl;
36     cout << "Nodos esperados: 6" << endl;
37     cout << "Nodos calculados: " << contarNodos(raiz) << endl;
38
39     return 0;
40 }
```

Ejercicio 2



```
1  #include <iostream>
2  using namespace std;
3
4  struct Nodo {
5      int valor;
6      Nodo* izquierdo;
7      Nodo* derecho;
8      Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
9  };
10
11  Nodo* insertar(Nodo* raiz, int valor) {
12      // TODO: Implementa tu lógica aquí
13      if (raiz == nullptr) return new Nodo(valor);
14      if (valor < raiz->valor)
15          raiz->izquierdo = insertar(raiz->izquierdo, valor);
16      else
17          raiz->derecho = insertar(raiz->derecho, valor);
18      return raiz;
19  }
20
21  int main() {
22      Nodo* raiz = new Nodo(10);
23
24      insertar(raiz, 5);
25      insertar(raiz, 15);
26      insertar(raiz, 3);
27
28      cout << "--- Prueba Ejercicio 2 ---" << endl;
29      cout << "Raiz (Esperado 10): " << raiz->valor << endl;
30
31      if(raiz->izquierdo) cout << "Hijo Izquierdo (Esperado 5): " << raiz->izquierdo->valor << endl;
32      else cout << "Hijo Izquierdo: null" << endl;
33
34      if(raiz->derecho) cout << "Hijo Derecho (Esperado 15): " << raiz->derecho->valor << endl;
35      else cout << "Hijo Derecho: null" << endl;
36
37      if(raiz->izquierdo && raiz->izquierdo->izquierdo)
38          cout << "Hijo Izq del 5 (Esperado 3): " << raiz->izquierdo->izquierdo->valor << endl;
39      else cout << "Hijo Izq del 5: null" << endl;
40
41      return 0;
42  }
```

Ejercicio 3

```
1  #include <iostream>
2  #include <algorithm> // util para std::max
3  using namespace std;
4
5  // Asumiendo la estructura Nodo del Ejercicio 2
6  struct Nodo {
7      int valor;
8      Nodo* izquierdo;
9      Nodo* derecho;
10     Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
11 };
12
13 int calcularAltura(Nodo* raiz) {
14     // TODO: Implementa tu lógica aquí
15     if (raiz == nullptr) return 0;
16     return 1 + max(calcularAltura(raiz->izquierdo), calcularAltura(raiz->derecho));
17 }
18
19 int main() {
20     Nodo* raiz = new Nodo(1);
21     raiz->derecho = new Nodo(2);
22     raiz->derecho->izquierdo = new Nodo(3);
23
24     cout << "--- Prueba Ejercicio 3 ---" << endl;
25     cout << "Altura esperada: 3" << endl;
26     cout << "Altura calculada: " << calcularAltura(raiz) << endl;
27     cout << "Altura de arbol nulo (esperado 0): " << calcularAltura(nullptr) << endl;
28
29     return 0;
30 }
```

Ejercicio 4



```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  struct Nodo {
6      int valor;
7      Nodo* izquierdo;
8      Nodo* derecho;
9      Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
10 };
11
12 void inOrderAux(Nodo* nodo, vector<int>& resultado) {
13     // TODO: Implementa tu logica de recorrido aqui
14     if (nodo == nullptr) return;
15     inOrderAux(nodo->izquierdo, resultado);
16     resultado.push_back(nodo->valor);
17     inOrderAux(nodo->derecho, resultado);
18 }
19
20 vector<int> recorridoInOrder(Nodo* raiz) {
21     vector<int> resultado;
22     inOrderAux(raiz, resultado);
23     return resultado;
24 }
25
26 int main() {
27     Nodo* raiz = new Nodo(4);
28     raiz->izquierdo = new Nodo(2);
29     raiz->derecho = new Nodo(6);
30     raiz->izquierdo->izquierdo = new Nodo(1);
31     raiz->izquierdo->derecho = new Nodo(3);
32     raiz->derecho->izquierdo = new Nodo(5);
33     raiz->derecho->derecho = new Nodo(7);
34
35     vector<int> res = recorridoInOrder(raiz);
36
37     cout << "--- Prueba Ejercicio 4 ---" << endl;
38     cout << "Resultado esperado: 1 2 3 4 5 6 7 " << endl;
39     cout << "Tu resultado:          ";
40     for(int val : res) {
41         cout << val << " ";
42     }
43     cout << endl;
44
45     return 0;
46 }
```

Ejercicio 5



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: MARZO – JULIO 2026



```
1  #include <iostream>
2  using namespace std;
3
4  struct Nodo {
5      int valor;
6      Nodo* izquierdo;
7      Nodo* derecho;
8      Nodo(int v) : valor(v), izquierdo(nullptr), derecho(nullptr) {}
9  };
10
11  Nodo* invertir(Nodo* raiz) {
12      // TODO: Implementa tu lógica aquí
13      if (raiz == nullptr) return nullptr;
14      Nodo* temp = raiz->izquierdo;
15      raiz->izquierdo = invertir(raiz->derecho);
16      raiz->derecho = invertir(temp);
17      return raiz;
18  }
19
20  int main() {
21      Nodo* raiz = new Nodo(1);
22      raiz->izquierdo = new Nodo(2);
23      raiz->derecho = new Nodo(3);
24
25      cout << "--- Prueba Ejercicio 5 ---" << endl;
26      cout << "Antes de invertir:" << endl;
27      cout << "Hijo Izq: " << raiz->izquierdo->valor << " | Hijo Der: " << raiz->derecho->valor << endl;
28
29      invertir(raiz);
30
31      cout << "\nDespues de invertir (Esperado: Izq 3 | Der 2):" << endl;
32
33      if(raiz->izquierdo) cout << "Hijo Izq: " << raiz->izquierdo->valor;
34      else cout << "Hijo Izq: null";
35
36
37      if(raiz->derecho) cout << "Hijo Der: " << raiz->derecho->valor << endl;
38      else cout << "Hijo Der: null" << endl;
39
40
41      return 0;
42  }
```